Information-Theoretic Wordle Solver for Turkish

Thomas ACAR

August 2026

Abstract

This document explains the mathematical principles and algorithmic design behind the `Wordle TR` solver. Inspired by 3Blue1Brown's video on Solving Wordle using Information Theory, I built this solver to find the optimal guess at any stage of Turkish Wordle. This breakdown is written to be accessible and intuitive for readers of all backgrounds. It explains how Wordle works as an information game, defines Information Theory and Shannon Entropy using clear step-by-step analogies, and demonstrates why searching the full valid guess vocabulary out-performs guessing only remaining candidate words.

1 Introduction

Wordle is a simple 5-letter word puzzle, but under the hood, it is a classic problem in **Information Theory**. After watching 3Blue1Brown's video on how entropy can be used to solve Wordle mathematically, I wanted to apply the same concept to Turkish Wordle.

The goal of this project was to build a fast, reactive web application that evaluates every valid word, calculates how much information it is expected to give, and recommends the move that narrows down the remaining secret words as fast as possible.

—

2 Understanding Wordle as an Information Game

2.1 The Rules and Feedback Signal

In Wordle, the game distinguishes between two word lists:

- **Secret Solution Answers ($\mathcal{A}$)**: The curated set of possible target solution words ($|\mathcal{A}| = 2936$ unique words).
- **Valid Guess Vocabulary ($\mathcal{V}$)**: The broader dictionary of all valid words accepted as guesses ($|\mathcal{V}| = 5500$ unique words).

In each turn, you guess a 5-letter word $g \in \mathcal{V}$. The game responds with 5 colored tiles (a feedback pattern p):

- **Green (2)**: Correct letter in the exact correct position.
- **Yellow (1)**: Letter exists in the secret word, but in a different position.
- **Gray (0)**: Letter does not appear in the secret word (or matched elsewhere).

Since there are 3 possible colors for each of the 5 tiles, there are at most $3^5 = 243$ possible color feedback patterns for any guess.

2.2 Candidate Filtering ($\mathcal{C}_k$)

At the start of the game (Turn 1), the secret word could be any of the 2,936 target solution words in $\mathcal{A}$. We call this set of possibilities the **Candidate Set** ($\mathcal{C}_1 = \mathcal{A}$).

When you make a guess $g_1 \in \mathcal{V}$ and receive color pattern p_1, you eliminate every word in $\mathcal{C}_1$ that would not produce pattern p_1. The remaining valid target words form a smaller candidate set $\mathcal{C}_2$.

At turn k, your goal is to shrink the candidate set $\mathcal{C}_k$ down to exactly **1 remaining word** as quickly as possible.

—

3 The Math Behind the Magic: Information Theory

3.1 What is Information?

Imagine playing a game of "20 Questions" to guess an animal:

- Asking *"Is it a mammal?"* splits the remaining animals roughly 50/50. A "Yes" or "No" cuts your choices in half. That gives you a lot of information.
- Asking *"Is it a flamingo?"* on Question 1 is risky. 99.9% of the time the answer is "No", which barely eliminates anything.

In Information Theory, **information measures the reduction of uncertainty**. A good move is one that guarantees a large reduction in remaining possibilities, no matter what answer the game gives you.

3.2 What is a "Bit"? ($\log_2$)

Before jumping into formulas, let us build a clear, intuitive picture of what a **bit** actually is and where the base-2 logarithm ($\log_2$) comes from.

- **The Fundamental Unit: A Single Yes/No Question**
  At its most basic level, **one bit** is the amount of information you gain when you get the answer to a single, perfectly balanced **Yes/No question**.

  Imagine I am thinking of a number between 1 and 2:

  "Is the number 1?"

  If the two choices were equally likely, receiving the answer ("Yes" or "No") instantly resolves all your uncertainty. That single answer gives you **1 bit of information**. It reduces 2 possibilities down to 1.

- **Stacking Yes/No Questions**
  What happens if we ask multiple Yes/No questions in a row?
  - **1 Yes/No Question (1 bit)**: Can distinguish between $2^1 = \mathbf{2}$ possibilities.
  - **2 Yes/No Questions (2 bits)**: The first question splits the space into 2 groups; the second question splits each of those in half again. Together, they distinguish between $2 \times 2 = 2^2 = \mathbf{4}$ possibilities.
  - **3 Yes/No Questions (3 bits)**: Can distinguish between $2 \times 2 \times 2 = 2^3 = \mathbf{8}$ possibilities.
  - b **Yes/No Questions (b bits)**: Can distinguish between 2^b possibilities.

Notice the pattern: every additional bit of information **doubles** the number of possibilities you can eliminate or distinguish between.

- **Going Backwards: Where $\log_2$ Comes From**
  Now suppose we flip the question around. Instead of asking how many possibilities b bits can handle, suppose we already know we have N possibilities, and we want to know:

  "How many Yes/No questions (bits) do I need to pinpoint 1 item out of N?"

  We need to solve the equation: $2^b = N$. To solve for the exponent b, we take the logarithm base 2 of both sides:

  $$b = \log_2(N) \tag{1}$$

  That is all a base-2 logarithm is! $\log_2(N)$ **simply counts how many fair Yes/No questions you must ask to isolate 1 option out of N.**

- **Applying This to Wordle**
  In our Turkish Wordle game, we start with $N = 2,936$ candidate solution words. How much information do we need to collect to identify the single secret word?

  $$\text{Total Information Needed} = \log_2(2936) \approx \mathbf{11.52 \ \ bits} \tag{2}$$

  This means that finding the secret word out of 2,936 target candidates is mathematically equivalent to getting the answers to roughly **11.52 ideal Yes/No questions**. Every guess we make in Wordle is an attempt to ask a question that retrieves as many of these 11.52 bits as possible in one turn.

3.3 Probability Buckets

When you test a guess $g \in \mathcal{V}$ against all remaining candidate words $\mathcal{C}_k \subseteq \mathcal{A}$, each candidate word will fall into one of the 243 possible color pattern "buckets".

For example, if 100 candidate words remain:

- Bucket A (pattern `02010`: 1 green, 1 yellow, 3 grays) receives 50 words ($P = \frac{50}{100} = 0.50$).
- Bucket B (pattern `22000`: 2 greens, 3 grays) receives 25 words ($P = \frac{25}{100} = 0.25$).
- Bucket C (pattern `10202`: 2 greens, 1 yellow, 2 grays) receives 25 words ($P = \frac{25}{100} = 0.25$).

If a pattern bucket p contains N_p words, receiving pattern p provides:

$$I(p) = \log_2\left(\frac{|\mathcal{C}_k|}{N_p}\right) = -\log_2 P(p) \quad \text{bits of information.} \tag{3}$$

Notice that **smaller buckets give more bits of information** (if a bucket has only 1 word, $I = \log_2(100/1) = 6.64$ bits, and you win immediately!).

3.4 Shannon Entropy ($H(g)$)

When you type a guess into Wordle, you cannot know in advance which specific color pattern the game will return. One pattern might leave 50 candidates, while another might leave only 2 candidates.

To evaluate how good a guess is before playing it, we need a way to measure its overall **expected information gain**.

Why a Weighted Average?

If a guess has multiple possible outcomes, its overall value is the **weighted average** of the information gained from each outcome, where each outcome's value is weighted by the probability of it actually happening:

$$\text{Expected Information} = \sum(\text{Probability of Pattern } p) \times (\text{Information Gained from Pattern } p)$$

Why is there a Minus Sign ($-$) in the Formula?

Recall from Section 3.3 that the information gained from a pattern p with probability $P(p)$ is defined as:

$$I(p) = \log_2\left(\frac{1}{P(p)}\right) = -\log_2 P(p) \tag{4}$$

Because $P(p)$ is a fraction between 0 and 1 (for example, $P(p) = 0.25$), its logarithm $\log_2(0.25) = -2$ is always a **negative number**.

However, information measures a positive reduction in uncertainty (we gain $+2$ bits of knowledge, not -2 bits). To flip the negative logarithm into a positive information quantity, we place a **minus sign** ($-$) in front of the logarithm.

The Shannon Entropy Formula

Combining the probability weighting $P(p)$ with the positive information gain $-\log_2 P(p)$ gives the mathematical definition of **Shannon Entropy**:

$$H(g \mid \mathcal{C}_k) = -\sum_p P(p)\log_2 P(p) \tag{5}$$

Shannon Entropy represents the expected information (in bits) that playing guess g will yield across all possible color feedback patterns.

A Worked Example

In our 100-word example from Section 3.3 with outcome probabilities $P = [0.50, 0.25, 0.25]$:

$$\begin{aligned} H(g) &= -\left[0.50\log_2(0.50) + 0.25\log_2(0.25) + 0.25\log_2(0.25)\right] \\ &= -\left[0.50(-1) + 0.25(-2) + 0.25(-2)\right] \\ &= 0.50 + 0.50 + 0.50 = \mathbf{1.50} \text{ bits} \end{aligned}$$

On average, playing guess g will reduce the 100 remaining words down to $\frac{100}{2^{1.50}} \approx 35$ words. A higher entropy value means a better guess.

—

4 Why Full Dictionary Search Beats Candidate-Only Search

A common instinct is to guess only words that could still be the secret answer ($\mathcal{C}_k$). However, searching the **full valid guess vocabulary** $\mathcal{V}$ (including words that cannot be target answers) is often much more powerful.

4.1 The _AKER Trap Example

Suppose 4 candidates remain:

$$\mathcal{C}_k = \{\text{BAKER}, \text{FAKER}, \text{MAKER}, \text{TAKER}\}$$

- **Guessing from Candidates** (e.g., $g = \text{BAKER}$):
  - $\frac{1}{4}$ chance it is correct (all green).
  - $\frac{3}{4}$ chance it is wrong (1 gray, 4 green), leaving 3 words behind.
  - Expected Entropy: **0.81 bits**.
- **Guessing from Full Dictionary** (e.g., a word containing B, F, M, T):
  - Tests 4 letters at once. Every candidate produces a unique pattern ($\frac{1}{4}$ probability each).
  - Expected Entropy: **2.0 bits** ($\log_2(4) = 2.0$).
  - **Result**: Guarantees finding the secret word on the next turn 100% of the time.

4.2 Tie-Breaking Candidate Bonus

When an external word $g_{\text{ext}} \in \mathcal{V} \setminus \mathcal{C}_k$ and a candidate word $g_{\text{cand}} \in \mathcal{C}_k$ produce equal entropy, we give a slight preference to g_{cand} since it has a chance of winning immediately:

$$S(g) = H(g \mid \mathcal{C}_k) + \epsilon \cdot \mathbb{I}(g \in \mathcal{C}_k) \tag{6}$$

where $\epsilon = 0.0001$.

—

5 Algorithm Pseudocode

Algorithm 1 Feedback Pattern Matcher $f(g, w)$

Input: Guess word g, Secret candidate w
Initialize pattern array $p \leftarrow [0, 0, 0, 0, 0]$
▷ **Phase 1: Mark Greens (exact matches)**
for $i \leftarrow 0$ **to** 4 **do**
    if $g[i] = w[i]$ **then**
        $p[i] \leftarrow 2$; mark $g[i]$ and $w[i]$ as matched
    end if
end for
▷ **Phase 2: Mark Yellows (partial matches)**
for $i \leftarrow 0$ **to** 4 **do**
    if $g[i]$ unmatched **and** $g[i] \in w$ (unmatched) **then**
        $p[i] \leftarrow 1$; mark matched letter in w
    end if
end for
return $p_0 p_1 p_2 p_3 p_4$

Algorithm 2 Optimal Best Word Finder findBestWord($\mathcal{C}_k, \mathcal{V}$)

Input: Target Candidates $\mathcal{C}_k \subseteq \mathcal{A}$, Valid Guess Vocabulary $\mathcal{V}$
if $|\mathcal{C}_k| \leq 2$ **then**
    return $\arg\max_{g \in \mathcal{C}_k} H(g \mid \mathcal{C}_k)$
end if
$S_{\max} \leftarrow -1$, $g^* \leftarrow \mathcal{C}_k[0]$, $H^* \leftarrow 0$
for each $g \in \mathcal{V}$ **do**
    Compute $H(g \mid \mathcal{C}_k)$ using Equation (3)
    $S(g) \leftarrow H(g \mid \mathcal{C}_k) + (0.0001 \text{ if } g \in \mathcal{C}_k \text{ else } 0)$
    if $S(g) > S_{\max}$ **then**
        $S_{\max} \leftarrow S(g)$; $g^* \leftarrow g$; $H^* \leftarrow H(g \mid \mathcal{C}_k)$
    end if
end for
return {word: g^*, entropy: H^*}

—

6 Empirical Results: Top Turkish Starting Words

Evaluating all 5,500 valid guess words in $\mathcal{V}$ against all 2,936 target solution answers in $\mathcal{A}$ yields the top opening words. As detailed in Table 1, **TARİK**, **MERAK**, and **KENAR** achieve the highest information gain across the entire dataset, solving any target word in an average of just 3.58 turns.

Table 1: Top Optimal Turkish Starting Words

Rank	Word	Shannon Entropy (Bits)	Avg. Guesses ($\mathbb{E}[N]$)	Space Reduction
1	**TARİK**	**5.8083**	**3.58 turns**	**97.1%** (2936 $\rightarrow \approx$ 52)
2	**MERAK**	**5.8057**	**3.58 turns**	**97.0%** (2936 $\rightarrow \approx$ 53)
3	**KENAR**	**5.7868**	**3.59 turns**	**97.0%** (2936 $\rightarrow \approx$ 53)
4	**SERAK**	5.7801	—	96.9%
5	**KAMER**	5.7785	—	96.9%
6	**KELAM**	5.7379	—	96.5%
7	**KALEM**	5.7162	—	96.4%
8	**TALİK**	5.7081	—	96.4%
9	**KETAL**	5.6973	—	96.3%
10	**KEMAL**	5.6883	—	96.3%

7 Conclusion

By leveraging Information Theory and Shannon Entropy maximization across the full valid vocabulary $\mathcal{V}$, the `Wordle TR` solver reduces thousands of possibilities down to the single secret word in an average of under 3.6 turns.